

Project: Data Transformer

Objective:

"Data Transformer" is a comprehensive SQL project designed to enhance your practical knowledge of advanced SQL operations. This project will guide students through working with Joins, Subqueries, Date and String Manipulation Functions, Window Functions, and the SQL CASE Expression. By completing this project, students will build the skills needed to transform and manipulate data for reporting, analysis, and complex queries.

Project Overview:

The project simulates a Corporate Data Analysis System with the following core functionalities:

- 1. Customer Information Management**
- 2. Sales Transaction Processing**
- 3. Employee Performance Data**

1 Customers Table

Fields:

- CustomerID
- FirstName
- LastName
- Email
- RegistrationDate

Sample Data:

CustomerID	FirstName	LastName	Email	RegistrationDate
1	John	Doe	john.doe@email.com	2022-03-15
2	Jane	Smith	jane.smith@email.com	2021-11-02

2 Orders Table

Fields:

- OrderID
- CustomerID
- OrderDate
- TotalAmount

Sample Data:

OrderID	CustomerID	OrderDate	TotalAmount
101	1	2023-07-01	150.50
102	2	2023-07-03	200.75

3 Employees Table**Fields:**

- EmployeeID
- FirstName
- LastName
- Department
- HireDate
- Salary

Sample Data:

EmployeeID	FirstName	LastName	Department	HireDate	Salary
1	Mark	Johnson	Sales	2020-01-15	50000.00
2	Susan	Lee	HR	2021-03-20	55000.00

Queries to Perform:

1. **INNER JOIN:** Retrieve all orders and customer details where orders exist.
2. **LEFT JOIN:** Retrieve all customers and their corresponding orders (if any).
3. **RIGHT JOIN:** Retrieve all orders and their corresponding customers (if any).
4. **FULL OUTER JOIN:** Retrieve all customers and all orders, regardless of matching.
5. Subquery to find customers who have placed orders worth more than the average amount.
6. Subquery to find employees with salaries above the average salary.

Queries to Perform:

1. **INNER JOIN:** Retrieve all orders and customer details where orders exist.
2. **LEFT JOIN:** Retrieve all customers and their corresponding orders (if any).
3. **RIGHT JOIN:** Retrieve all orders and their corresponding customers (if any).
4. **FULL OUTER JOIN:** Retrieve all customers and all orders, regardless of matching.
5. Subquery to find customers who have placed orders worth more than the average amount.
6. Subquery to find employees with salaries above the average salary.
7. Extract the year and month from the OrderDate.
8. Calculate the difference in days between two dates (order date and current date).
9. Format the OrderDate to a more readable format (e.g., 'DD-MMM-YYYY').

10. Concatenate FirstName and LastName to form a full name:
11. Replace part of a string (e.g., replace 'John' with 'Jonathan'):
12. Convert FirstName to uppercase and LastName to lowercase:
13. Trim extra spaces from the Email field:
14. Calculate the running total of TotalAmount for each order.
15. Rank orders based on TotalAmount using the RANK() function:
16. Assign a discount based on TotalAmount in orders (e.g., > 1000: 10% off, > 500: 5% off).
17. Categorize employees' salaries as high, medium, or low.